

# JAVA WEB APPLICATION TECHNOLOGIES



# What is JSP

- **JSP** technology is used to create web application just like Servlet technology. It can be thought of as an extension to Servlet because it provides more functionality than servlet such as expression language, JSTL, etc.
- A JSP page consists of HTML tags and JSP tags. The JSP pages are easier to maintain than Servlet because we can separate designing and development. It provides some additional features such as Expression Language, Custom Tags, etc.



# Advantages of JSP

## 1) Extension to Servlet

- JSP technology is the extension to Servlet technology. We can use all the features of the Servlet in JSP. In addition to, we can use implicit objects, predefined tags, expression language and Custom tags in JSP, that makes JSP development easy.

## 2) Easy to maintain

- JSP can be easily managed because we can easily separate our business logic with presentation logic. In Servlet technology, we mix our business logic with the presentation logic.

## 3) Fast Development: No need to recompile and redeploy

- If JSP page is modified, we don't need to recompile and redeploy the project. The Servlet code needs to be updated and recompiled if we have to change the look and feel of the application.

## 4) Less code than Servlet

- In JSP, we can use many tags such as action tags, JSTL, custom tags, etc. that reduces the code. Moreover, we can use EL, implicit objects, etc.



# JSP Scriptlet tag (Scripting elements)

- The scripting elements provides the ability to insert java code inside the jsp. There are three types of scripting elements:
  - scriptlet tag
  - expression tag
  - declaration tag



# JSP scriptlet tag

- A scriptlet tag is used to execute java source code in JSP. Syntax is as follows:

```
<% java source code %>
```

```
<%  
int a=Integer.parseInt(request.getParameter("num1"));  
int b=Integer.parseInt(request.getParameter("num2"));  
int sum=a+b;  
out.println("sum : "+sum);  
%>
```



# JSP expression tag

- The code placed within **JSP expression tag** is *written to the output stream of the response*. So you need not write `out.print()` to write data. It is mainly used to print the values of variable or method.
- Syntax of JSP expression tag

```
<%= statement %>
```

```
Total is :<%=sum %>
```



# JSP Declaration Tag

- The **JSP declaration tag** is used *to declare fields and methods*.
- The code written inside the jsp declaration tag is placed outside the service() method of auto generated servlet.
- The syntax of the declaration tag is as follows:

```
<%! field or method declaration %>
```

```
<%!  
int k=5;  
%>
```



# Difference between JSP Scriptlet tag and Declaration tag

| Jsp Scriptlet Tag                                                                        | Jsp Declaration Tag                                                                             |
|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| The jsp scriptlet tag can only declare variables not methods.                            | The jsp declaration tag can declare variables as well as methods.                               |
| The declaration of scriptlet tag is placed inside the <code>_jspService()</code> method. | The declaration of jsp declaration tag is placed outside the <code>_jspService()</code> method. |





# JSP Implicit Objects

- There are **9 jsp implicit objects**. These objects are *created by the web container* that are available to all the jsp pages.

| Object      | Type                |
|-------------|---------------------|
| out         | JspWriter           |
| request     | HttpServletRequest  |
| response    | HttpServletResponse |
| config      | ServletConfig       |
| application | ServletContext      |
| session     | HttpSession         |
| pageContext | PageContext         |
| page        | Object              |
| exception   | Throwable           |



# JSP out implicit object

- For writing any data to the buffer, JSP provides an implicit object named out. It is the object of JspWriter. In case of servlet you need to write:

```
PrintWriter out=response.getWriter();
```

- But in JSP, you don't need to write this code.

```
<html>
<body>
<% out.print("Today is:"+java.util.Calendar.getInstance().getTime()); %>
</body>
</html>
```



# JSP request implicit object

- The **JSP request** is an implicit object of type `HttpServletRequest` i.e. created for each jsp request by the web container. It can be used to get request information such as parameter, header information, remote address, server name, server port, content type, character encoding etc.
- It can also be used to set, get and remove attributes from the jsp request scope.
- Let's see the simple example of request implicit object where we are printing the sum of two numbers



# JSP request implicit object

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<form action="add.jsp">
first number :<input type="text" name="num1"><br>
Second number :<input type="text" name="num2"><br>
<input type="submit" value="calculate">
</form>
</body>
</html>
```

home.jsp for  
entering  
number

Add.jsp for adding  
numbers

```
<%@page import="java.util.Date"%>
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<%
int a=Integer.parseInt(request.getParameter("num1"));
int b=Integer.parseInt(request.getParameter("num2"));
int sum=a+b;
out.println("sum :"+sum);
%>
</body>
</html>
```



# JSP response implicit object

- In JSP, response is an implicit object of type `HttpServletResponse`. The instance of `HttpServletResponse` is created by the web container for each jsp request.
- It can be used to add or manipulate response such as redirect response to another resource, send error etc.
- Let's see the example of response implicit object where we are redirecting the response to the Google.



# JSP response implicit object

index.html

```
<form action="welcome.jsp">  
<input type="text" name="uname">  
<input type="submit" value="go"> <br/>  
</form>
```

welcome.jsp

```
<%  
response.sendRedirect("http://www.google.com");  
%>
```



# JSP config implicit object

- In JSP, config is an implicit object of type *ServletConfig*. This object can be used to get initialization parameter for a particular JSP page. The config object is created by the web container for each jsp page.
- Generally, it is used to get initialization parameter from the web.xml file.

Web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://xmlns.jcp.org/xml/ns/javaee" xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-app_3_2.xsd" id="WebApp_ID" version="3.0">
  <display-name>JSP project</display-name>

  <servlet>
    <servlet-name>adding</servlet-name>
    <jsp-file>/add.jsp</jsp-file>
    <init-param>
      <param-name>constant</param-name>
      <param-value>3.14</param-value>
    </init-param>
  </servlet>
  <servlet-mapping>
    <servlet-name>adding</servlet-name>
    <url-pattern>/add</url-pattern>
  </servlet-mapping>
</web-app>
```

# JSP config implicit object

```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6 <meta charset="ISO-8859-1">
7 <title>Insert title here</title>
8 </head>
9 <body>
10 <form action="add">
11   first number :<input type="text" name="num1"><br>
12   Second number :<input type="text" name="num2"><br>
13   <input type="submit" value="calculate">
14 </form>
15 </body>
16 </html>
```

home.jsp

Add.js  
p

Config  
object

```
1 <%@page import="java.util.Date"%>
2 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
3   pageEncoding="ISO-8859-1"%>
4 <!DOCTYPE html>
5 <html>
6 <head>
7 <meta charset="ISO-8859-1">
8 <title>Insert title here</title>
9 </head>
10 <body>
11 <%
12   int a=Integer.parseInt(request.getParameter("num1"));
13   int b=Integer.parseInt(request.getParameter("num2"));
14   int sum=a+b;
15   out.println("sum :"+sum);
16   double cons=Double.parseDouble(config.getInitParameter("constant"));
17   out.println("our constant is:"+cons);
18 %>
19
```

Adva





# JSP application implicit object

- In JSP, application is an implicit object of type *ServletContext*.
- The instance of *ServletContext* is created only once by the web container when application or project is deployed on the server.
- This object can be used to get initialization parameter from configuration file (web.xml). It can also be used to get, set or remove attribute from the application scope.
- This initialization parameter can be used by all jsp pages.



# JSP application implicit object

Web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://xmlns.jcp.org/xml/ns/javaee" xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee/web-app_3_0.xsd">
  <display-name>JSP project</display-name>

  <servlet>
    <servlet-name>adding</servlet-name>
    <jsp-file>/add.jsp</jsp-file>
    <init-param>
      <param-name>constant</param-name>
      <param-value>3.14</param-value>
    </init-param>
  </servlet>

  <servlet-mapping>
    <servlet-name>adding</servlet-name>
    <url-pattern>/add</url-pattern>
  </servlet-mapping>

  <context-param>
    <param-name>website</param-name>
    <param-value>www.rca.gov.rw</param-value>
  </context-param>
</web-app>
```

Application  
object

```
<%@page import="java.util.Date"%>
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<%
int a=Integer.parseInt(request.getParameter("num1"));
int b=Integer.parseInt(request.getParameter("num2"));
int sum=a+b;
out.println("sum :"+sum);
double cons=Double.parseDouble(config.getInitParameter("constant"));
String website=application.getInitParameter("website");
out.println("our constant is:"+cons);
out.println("our website:"+website);
%>
```



# session implicit object

- In JSP, session is an implicit object of type HttpSession. The Java developer can use this object to set, get or remove attribute or to get session information.

```
<%@page import="java.util.Date"%>
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<%
int a=Integer.parseInt(request.getParameter("num1"));
int b=Integer.parseInt(request.getParameter("num2"));
int sum=a+b;
out.println("sum : "+sum);
double cons=Double.parseDouble(config.getInitParameter("constant"));
String website=application.getInitParameter("website");
session.setAttribute("sum", sum);

%>

</body>
</html>
```

Sum is set into  
session object

# Example of session object

```
1 <%@page import="java.util.Date"%>
2 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
3   pageEncoding="ISO-8859-1"%>
4 <!DOCTYPE html>
5 <html>
6 <head>
7 <meta charset="ISO-8859-1">
8 <title>Insert title here</title>
9 </head>
10 <body>
11 <%
12   int a=Integer.parseInt(request.getParameter("num1"));
13   int b=Integer.parseInt(request.getParameter("num2"));
14   int sum=a+b;
15   out.println("sum :"+sum);
16   double cons=Double.parseDouble(config.getInitParameter("constant"));
17   String website=application.getInitParameter("website");
18   session.setAttribute("sum", sum);
19   pageContext.setAttribute("sum",sum,PageContext.SESSION_SCOPE);
20 %>
21
22 </body>
23 </html>
```

Session scope

Session  
scope

```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6 <meta charset="ISO-8859-1">
7 <title>Insert title here</title>
8 </head>
9 <body>
10 <%
11   String sum=(String)session.getAttribute("sum");
12   String sum2=(String)pageContext.getAttribute("sum",PageContext.SESSION_
13 %>
14 The sum: <%=sum2 %>
15 </body>
16 </html>
```



# session implicit object

- We have result.jsp to get the from the session object

```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6 <meta charset="ISO-8859-1">
7 <title>Insert title here</title>
8 </head>
9 <body>
10 <%
11 String sum=(String)session.getAttribute("sum");
12 %>
13 The sum: <%=sum %>
14 </body>
15 </html>
```



# pageContext implicit object

- In JSP, pageContext is an implicit object of type PageContext class. The pageContext object can be used to set, get or remove attribute from one of the following scopes:
  - page
  - request
  - session
  - application



# Example of pageContext implicit object

```
<%@page import="java.util.Date"%>
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<%
    int a=Integer.parseInt(request.getParameter("num1"));
    int b=Integer.parseInt(request.getParameter("num2"));
    int sum=a+b;
    out.println("sum :"+sum);
    int cons=Integer.parseInt(config.getInitParameter("constant"));
    String website=application.getInitParameter("website");
    session.setAttribute("sum", sum);
    pageContext.setAttribute("sum",sum,PageContext.SESSION_SCOPE);
%>
```

Pagecontext in  
session scope



# exception implicit object

- In JSP, exception is an implicit object of type `java.lang.Throwable` class. This object can be used to print the exception. But it can only be used in error pages. It is better to learn it after page directive.





# JSP directives

- The **jsp directives** are messages that tells the web container how to translate a JSP page into the corresponding servlet.
- There are three types of directives:
  - page directive
  - include directive
  - taglib directive



# JSP - The page Directive

- The **page** directive is used to provide instructions to the container. These instructions pertain to the current JSP page. You may code page directives anywhere in your JSP page. By convention, page directives are coded at the top of the JSP page.

```
<%@ page attribute = "value" %>
```



# Attributes of JSP page directive

| S.No. | Attribute & Purpose                                                                                            |
|-------|----------------------------------------------------------------------------------------------------------------|
| 1     | <b>buffer</b><br>Specifies a buffering model for the output stream.                                            |
| 2     | <b>autoFlush</b><br>Controls the behavior of the servlet output buffer.                                        |
| 3     | <b>contentType</b><br>Defines the character encoding scheme.                                                   |
| 4     | <b>errorPage</b><br>Defines the URL of another JSP that reports on Java unchecked runtime exceptions.          |
| 5     | <b>isErrorPage</b><br>Indicates if this JSP page is a URL specified by another JSP page's errorPage attribute. |

|    |                                                                                                                                 |
|----|---------------------------------------------------------------------------------------------------------------------------------|
| 6  | <b>extends</b><br>Specifies a superclass that the generated servlet must extend.                                                |
| 7  | <b>import</b><br>Specifies a list of packages or classes for use in the JSP as the Java import statement does for Java classes. |
| 8  | <b>info</b><br>Defines a string that can be accessed with the servlet's <b>getServletInfo()</b> method.                         |
| 9  | <b>isThreadSafe</b><br>Defines the threading model for the generated servlet.                                                   |
| 10 | <b>language</b><br>Defines the programming language used in the JSP page.                                                       |
| 11 | <b>session</b><br>Specifies whether or not the JSP page participates in HTTP sessions                                           |
| 12 | <b>isELIgnored</b><br>Specifies whether or not the EL expression within the JSP page will be ignored.                           |
| 13 | <b>isScriptingEnabled</b><br>Determines if the scripting elements are allowed for use.                                          |



# Jsp Include Directive

- The **include** directive is used to include a file during the translation phase. This directive tells the container to merge the content of other external files with the current JSP during the translation phase. You may code the **include** directives anywhere in your JSP page.
- The advantage of include directive is code reusability

Syntax of include directive

```
<%@ include file="resourceName" %>
```



# exception handling in jsp by specifying the error-page element in web.xml file

- This approach is better because you don't need to specify the `errorPage` attribute in each jsp page. Specifying the single entry in the web.xml file will handle the exception. In this case, either specify `exception-type` or `error-code` with the `location` element. If you want to handle all the exception, you will have to specify the `java.lang.Exception` in the `exception-type` element.

```
<error-page>
  <exception-type>java.lang.Exception</exception-type>
  <location>/error.jsp</location>
</error-page>
```



# The taglib Directive

- The JavaServer Pages API allow you to define custom JSP tags that look like HTML or XML tags and a tag library is a set of user-defined tags that implement custom behavior.
- The **taglib** directive declares that your JSP page uses a set of custom tags, identifies the location of the library, and provides means for identifying the custom tags in your JSP page.

```
<%@ taglib uri = "uri" prefix = "prefixOfTag" >
```

- Where, the **uri** attribute value resolves to a location the container understands and the **prefix** attribute informs a container what bits of markup are custom actions.



# JSP Action Tags

- These are server side tags which we can use instead of scriptlets
- There are many JSP action tags or elements, and each of them has its own uses and characteristics. Each JSP action tag is implemented to perform some precise tasks.
- The action tag is also implemented to streamline flow between pages and to employ a Java Bean.

Syntax:

Copy Code

```
<jsp:action_name attribute = "attribute_value" />
```



# List of JSP action

1. **jsp:forward**: is used for forwarding the request and response to other resources.
2. **jsp:include**: is used for including another resource.
3. **jsp:body**: is used for defining dynamically-defined body of XML element.
4. **jsp:useBean**: is used for creating or locating bean objects.
5. **jsp:setProperty**: is used for setting the value of property in the bean object.
6. **jsp:getProperty**: is used for printing the value of the property of the bean.
7. **jsp:element**: is used for defining XML elements dynamically.
8. **jsp:plugin**: is used for embedding other components (applets).
9. **jsp:param**: is used for setting the parameter for (forward or include) value.
10. **jsp:text**: is used for writing template text in JSP pages and documents.
11. **jsp:fallback**: is used for printing the message if plugins are working.
12. **jsp:attribute**: is used for defining attributes of dynamically-defined XML element.





# UseBean action

- This "useBean action": is used for creating or locating bean objects.

Copy Code

```
<jsp: useBean id="unique_name_to_identify_bean" class="package_name.class_name" />
```



# UseBean example

- This index page is taking student id, first name and last name from the user

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<form action="studentProcess.jsp">
  Student Identification Number<input type="text" name="studId"><br>
  Student First Name<input type="text" name="fName"><br>
  Student Last Name<input type="text" name="LName"><br>
  <input type="submit" value="save">
</form>

</body>
</html>
```



# UseBean example

- The studentProcess page, receives data from index page and display them

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<jsp:useBean id="stud" class="com.rca.Beans.Student" scope="session"></jsp:useBean>
<jsp:setProperty property="studId" name="stud" value='<%=request.getParameter("studId") %>' />
<jsp:setProperty property="fName" name="stud" value='<%=request.getParameter("fName") %>' />
<jsp:setProperty property="lName" name="stud" value='<%=request.getParameter("lName") %>' />
<h2>Student details</h2><br>
Student id:<jsp:getProperty property="studId" name="stud"/><br>
Student First Name:<jsp:getProperty property="fName" name="stud"/><br>
Student Last Name:<jsp:getProperty property="lName" name="stud"/><br>
</body>
</html>
```



# EL( Expression Language)

- Expression Language was introduced in JSP 2.0, it is a simplified language that is used to access and manipulate data within JSP pages also it provides a convenient way to access the data stored in objects like requests, sessions, applications, JavaBeans, etc.

## 1. Variable Access:

Used to access the value of a variable

```
${variableName}  
${xxxScope.variableName}
```

## 2. Property Access:

Used to access the value of a property

```
${object.property}
```

## 6. Accessing Request Parameters:

Accesses the value of a request parameter

```
${param. parametersName}
```

## 3. Arithmetic and Logical Operators:

Performs arithmetic or logical operations on numbers

```
${num1>num2}  
${num1+num2}
```

## 4. Conditional Expressions:

Evaluate a condition and return one of two values.

```
${condition ? value1 : value2 }
```

## 5. Collection Iterator :

Iterates over a collection and prints each item

```
<c:forEach var="item" items = "${collection}" >  
  ${item}  
</c:forEach >
```



# EL( Expression Language)

```
<%  
String name=request.getParameter("uname");  
out.println(name);  
%><br>
```



```
.4  
.5 hello,${param.uname}  
.6
```

# EL Syntax

- EL expressions are always given in curly braces, preceded by a dollar sign, i.e.

```
1  
2  
3 hello,${param.username}  
4  
5
```

- The first thing in an expression must be one of two things:
  - An attribute (in page, request, session or application scope)
  - An implicit object (not the same implicit objects available to scriptlets!)



# Accessing object properties in EL

- It has two methods for accessing object properties
  - Dot (.) operator
  - **[ ]** operator

```
${student.fName}<br>  
${student["phone"]}
```



# Dot operator

`${student.fName}`

Must be a **Map**  
or a **Bean**

**Map key**

OR

**Bean property**





# [ ] operator

```
List<String> district=Arrays.asList("kigali", "kicukiro","Nyarugenge");  
request.setAttribute("districts", district);  
RequestDispatcher rd=request.getRequestDispatcher("welcome.jsp");  
rd.forward(request, response);
```

```
<h2> Districts</h2>  
${requestScope.districts[1]}
```



# Implicit Objects in Expression Language (EL)

(these are different to JSP's implicit objects)

| Implicit Objects | Usage                                                                        |
|------------------|------------------------------------------------------------------------------|
| pageScope        | it maps the given attribute name with the value set in the page scope        |
| requestScope     | it maps the given attribute name with the value set in the request scope     |
| sessionScope     | it maps the given attribute name with the value set in the session scope     |
| applicationScope | it maps the given attribute name with the value set in the application scope |
| param            | it maps the request parameter to the single value                            |
| paramValues      | it maps the request parameter to an array of values                          |
| header           | it maps the request header name to the single value                          |
| headerValues     | it maps the request header name to an array of values                        |
| cookie           | it maps the given cookie name to the cookie value                            |
| initParam        | it maps the initialization parameter                                         |
| pageContext      | it provides access to many objects request, session etc.                     |



# Example of param implicit object

welcome.jsp ↗ welcomeProcess.jsp

```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6 <meta charset="ISO-8859-1">
7 <title>Insert title here</title>
8 </head>
9 <body>
10 <form action="welcomeProcess.jsp">
11 Your name<input type="text" name="uname"><br>
12 <input type="submit">
13 </form>
14 </body>
15 </html>
```

Welcome.  
jsp

welcomeProce  
ss.jsp

```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6 <meta charset="ISO-8859-1">
7 <title>Insert title here</title>
8 </head>
9 <body>
10 hello,${param.uname}
11
12 </body>
13 </html>
```



# Summary of EL implicit objects

| Name(s)                                                                                   | Description                                                    |
|-------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| <u>pageScope</u><br><u>requestScope</u><br><u>sessionScope</u><br><u>applicationScope</u> | Maps of attributes in each scope                               |
| <u>param</u><br><u>paramValues</u>                                                        | Maps of request parameters (i.e. GET and POST parameters)      |
| <u>header</u><br><u>headerValues</u>                                                      | Maps of request headers                                        |
| <u>cookie</u>                                                                             | Map of cookies                                                 |
| <u>pageContext</u>                                                                        | The actual <u>pageContext</u> object - for accessing anything! |



# EL operators

- EL provides some operators to provide basic functionality, like...

| Arithmetic     |   |     |
|----------------|---|-----|
| Addition       | + |     |
| Subtraction    | - |     |
| Multiplication | * |     |
| Division       | / | div |
| Remainder      | % | mod |

| Logical  |    |     |
|----------|----|-----|
| And      | && | and |
| Or       |    | or  |
| Negation | !  | not |

| Comparative            |    |           |
|------------------------|----|-----------|
| Equality               | == | <u>eq</u> |
| Not equals             | != | ne        |
| Less than              | <  | <u>lt</u> |
| Greater than           | >  | <u>gt</u> |
| Less than or equals    | <= | le        |
| Greater than or equals | >= | <u>ge</u> |



# EL is null friendly

- Web designers don't want to worry about things like `NullPointerException`
- Even if an attribute doesn't exist
  - EL displays nothing instead of an exception
  - If its used in an arithmetic expression it treats it as zero
- If you divide by zero using EL you get infinity instead of an exception



# JSTL (Java Standard Tag Library)

- The JavaServer Pages Standard Tag Library (JSTL) is a collection of useful JSP tags that encapsulates the core functionality common to many JSP applications.
- JSTL's tags are grouped into 5 areas (taglibs)



# JSTL Tags

| Tag Name        | Description                                                                                                                                                                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Core tags       | The JSTL core tag provide variable support, URL management, flow control, etc. The URL for the core tag is <b><a href="http://java.sun.com/jsp/jstl/core">http://java.sun.com/jsp/jstl/core</a></b> . The prefix of core tag is <b>c</b> .        |
| Function tags   | The functions tags provide support for string manipulation and string length. The URL for the functions tags is <b><a href="http://java.sun.com/jsp/jstl/functions">http://java.sun.com/jsp/jstl/functions</a></b> and prefix is <b>fn</b> .      |
| Formatting tags | The Formatting tags provide support for message formatting, number and date formatting, etc. The URL for the Formatting tags is <b><a href="http://java.sun.com/jsp/jstl/fmt">http://java.sun.com/jsp/jstl/fmt</a></b> and prefix is <b>fmt</b> . |
| XML tags        | The XML tags provide flow control, transformation, etc. The URL for the XML tags is <b><a href="http://java.sun.com/jsp/jstl/xml">http://java.sun.com/jsp/jstl/xml</a></b> and prefix is <b>x</b> .                                               |
| SQL tags        | The JSTL SQL tags provide SQL support. The URL for the SQL tags is <b><a href="http://java.sun.com/jsp/jstl/sql">http://java.sun.com/jsp/jstl/sql</a></b> and prefix is <b>sql</b> .                                                              |





# JSTL Core Tags

- The JSTL core tag provides variable support, URL management, flow control etc. The syntax used for including JSTL core library in your JSP is:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```



# JSTL Core Tags

## JSTL Core Tags List

| Tags                                                                      | Description                                                                                                                       |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>c:out</code>                                                        | It display the result of an expression, similar to the way <code>&lt;%=...%&gt;</code> tag work.                                  |
| <code>c:import</code>                                                     | It Retrives relative or an absolute URL and display the contents to either a String in 'var',a Reader in 'varReader' or the page. |
| <code>c:set</code>                                                        | It sets the result of an expression under evaluation in a 'scope' variable.                                                       |
| <code>c:remove</code>                                                     | It is used for removing the specified scoped variable from a particular scope.                                                    |
| <code>c:catch</code>                                                      | It is used for Catches any Throwable exceptions that occurs in the body.                                                          |
| <code>c:if</code>                                                         | It is conditional tag used for testing the condition and display the body content only if the expression evaluates is true.       |
| <code>c:choose</code> , <code>c:when</code> ,<br><code>c:otherwise</code> | It is the simple conditional tag that includes its body content if the evaluated condition is true.                               |
| <code>c:forEach</code>                                                    | It is the basic iteration tag. It repeats the nested body content for fixed number of times or over collection.                   |
| <code>c:forTokens</code>                                                  | It iterates over tokens which is separated by the supplied delimiters.                                                            |
| <code>c:param</code>                                                      | It adds a parameter in a containing 'import' tag's URL.                                                                           |
| <code>c:redirect</code>                                                   | It redirects the browser to a new URL and supports the context-relative URLs.                                                     |
| <code>c:url</code>                                                        | It creates a URL with optional query parameters.                                                                                  |



1/30/2026

# JSTL Function Tags

- The JSTL function provides a number of standard functions, most of these functions are common string manipulation functions. The syntax used for including JSTL function library in your JSP is:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
```



# JSTL Function Tags

| JSTL Functions                       | Description                                                                                       |                                   |                                                                                              |
|--------------------------------------|---------------------------------------------------------------------------------------------------|-----------------------------------|----------------------------------------------------------------------------------------------|
| <code>fn:contains()</code>           | It is used to test if an input string containing the specified substring in a program.            | <code>fn:split()</code>           | It splits the string into an array of substrings.                                            |
| <code>fn:containsIgnoreCase()</code> | It is used to test if an input string contains the specified substring as a case insensitive way. | <code>fn:toLowerCase()</code>     | It converts all the characters of a string to lower case.                                    |
| <code>fn:endsWith()</code>           | It is used to test if an input string ends with the specified suffix.                             | <code>fn:toUpperCase()</code>     | It converts all the characters of a string to upper case.                                    |
| <code>fn:escapeXml()</code>          | It escapes the characters that would be interpreted as XML markup.                                | <code>fn:substring()</code>       | It returns the subset of a string according to the given start and end position.             |
| <code>fn:indexOf()</code>            | It returns an index within a string of first occurrence of a specified substring.                 | <code>fn:substringAfter()</code>  | It returns the subset of string after a specific substring.                                  |
| <code>fn:trim()</code>               | It removes the blank spaces from both the ends of a string.                                       | <code>fn:substringBefore()</code> | It returns the subset of string before a specific substring.                                 |
| <code>fn:startsWith()</code>         | It is used for checking whether the given string is started with a particular string value.       | <code>fn:length()</code>          | It returns the number of characters inside a string, or the number of items in a collection. |
|                                      |                                                                                                   | <code>fn:replace()</code>         | It replaces all the occurrence of a string with another string sequence.                     |



# JSTL XML tags

- The JSTL XML tags are used for providing a JSP-centric way of manipulating and creating XML documents.
- The xml tags provide flow control, transformation etc. The url for the xml tags is **<http://java.sun.com/jsp/jstl/xml>** and prefix is x. The JSTL XML tag library has custom tags used for interacting with XML data. The syntax used for including JSTL XML tags library in your JSP is:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/xml" prefix="x" %>
```



# JSTL XML tags

| XML Tags                 | Descriptions                                                                                               |
|--------------------------|------------------------------------------------------------------------------------------------------------|
| <code>x:out</code>       | Similar to <code>&lt;%= ... &gt;</code> tag, but for XPath expressions.                                    |
| <code>x:parse</code>     | It is used for parse the XML data specified either in the tag body or an attribute.                        |
| <code>x:set</code>       | It is used to sets a variable to the value of an XPath expression.                                         |
| <code>x:choose</code>    | It is a conditional tag that establish a context for mutually exclusive conditional operations.            |
| <code>x:when</code>      | It is a subtag of that will include its body if the condition evaluated be 'true'.                         |
| <code>x:otherwise</code> | It is subtag of that follows tags and runs only if all the prior conditions evaluated be 'false'.          |
| <code>x:if</code>        | It is used for evaluating the test XPath expression and if it is true, it will processes its body content. |
| <code>x:transform</code> | It is used in a XML document for providing the XSL(Extensible Stylesheet Language) transformation.         |
| <code>x:param</code>     | It is used along with the transform tag for setting the parameter in the XSLT style sheet.                 |



# JSTL SQL Tags

- The JSTL sql tags provide SQL support. The url for the sql tags is **`http://java.sun.com/jsp/jstl/sql`** and prefix is **`sql`**.
- The SQL tag library allows the tag to interact with RDBMSs (Relational Databases) such as Microsoft SQL Server, MySQL, or Oracle. The syntax used for including JSTL SQL tags library in your JSP is:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql" %>
```



# JSTL SQL Tags

| SQL Tags                       | Descriptions                                                                               |
|--------------------------------|--------------------------------------------------------------------------------------------|
| <code>sql:setDataSource</code> | It is used for creating a simple data source suitable only for prototyping.                |
| <code>sql:query</code>         | It is used for executing the SQL query defined in its sql attribute or the body.           |
| <code>sql:update</code>        | It is used for executing the SQL update defined in its sql attribute or in the tag body.   |
| <code>sql:param</code>         | It is used for sets the parameter in an SQL statement to the specified value.              |
| <code>sql:dateParam</code>     | It is used for sets the parameter in an SQL statement to a specified java.util.Date value. |
| <code>sql:transaction</code>   | It is used to provide the nested database action with a common connection.                 |

